Financial Time Series Analysis

Lecture 13

- TA Session: Friday, May 7, 11:00am-12:00pm
- Final Exam: Monday, May 10, 10:00am
- Please take the course evaluation (for all classes)

Outline

- Details and Coverage of Final Exam
- Cointegration Example
- State Space Models
  - “Filtering in Finance”
  - Stochastic volatility model

Final Exam: 1

- The final exam will be comparable to the posted Time Series Exam, 90 minutes in length.
- It will be comprehensive, covering the entire course.
- It will consist largely of short answers (multiple choice, single and multiple answers) and a smaller number of problems requiring pen and ink.
- The test is closed book, notes and other sources as well except for a one-page (two-sided) cheat sheet. The use of computers/tablets, cell phones, calculators, etc. is forbidden, except for purposes of responding to, downloading and uploading the exam.
- No communication with others except the instructor, who will be available by zoom link.

Final Exam: 2

- Cell phones may be used to contact John Lehoczky during the exam for questions and technology problems as well as uploading any pen and ink solutions.
- Each student may have access to a 1-page (8.5x11, two-sided) “cheat sheet.” that can be in arbitrarily small font. It can be handwritten or computer generated.
- The final exam is not intended to be a test of memory. Rather it is intended to focus on an understanding of and the use of time series concepts.
- The final will be based on material covered in lectures, the course notes, and the homework. While no questions from homework will be replicated, homework and lecture should serve as a guide to doing well on the exam.

Final Exam Coverage: 1

- Rama Cont's stylized facts about financial markets
- Stationarity
  - Strong stationarity
  - Weak stationarity
  - Martingale Difference Sequences
- Autocovariance and autocorrelation functions
  - ACF, PACF -their definition and plots for a few simple time series like AR and MA
  - Bartlett's test (confidence limits for ACF plots for simple time series)
  - Ljung-Box test

Final Exam Coverage: 2

- Basic time series models (AR, MA, ARMA, ARIMA)
  - Know or be able to work out simple facts (mean, variance and covariance functions) for simple models.
- Transforming to achieve stationarity
  - Trend and Seasonal adjustments
  - Box-Cox transformation
  - Backshift (B) and ∇ operators and notation.
  - Differencing
  - Dickey Fuller test
- Parameter estimation using Yule Walker equations
- Forecasting AR, MA, ARMA models

Final Exam Coverage: 3

- ARCH/GARCH models
  - Understanding construction of ARCH/GARCH models
  - Tests to identify ARCH effects
    - Ljung-Box test
    - Engle's test
  - GARCH(p,q) and $GARCH^2(p,q)$ as ARMA
  - Forecasting GARCH

Final Exam Coverage: 4

- Multiple Time Series (VAR(p) process)
- Cointegration
  - Understanding the meaning of cointegration
  - Cointegration vs correlation
  - Tests for cointegration (Engle-Grange, Phillips-Ouliaris, Johansen)
- State Space Models
  - Their formulation
  - Understanding the “Kalman problems” (filtering, prediction and smoothing)

Cointegration

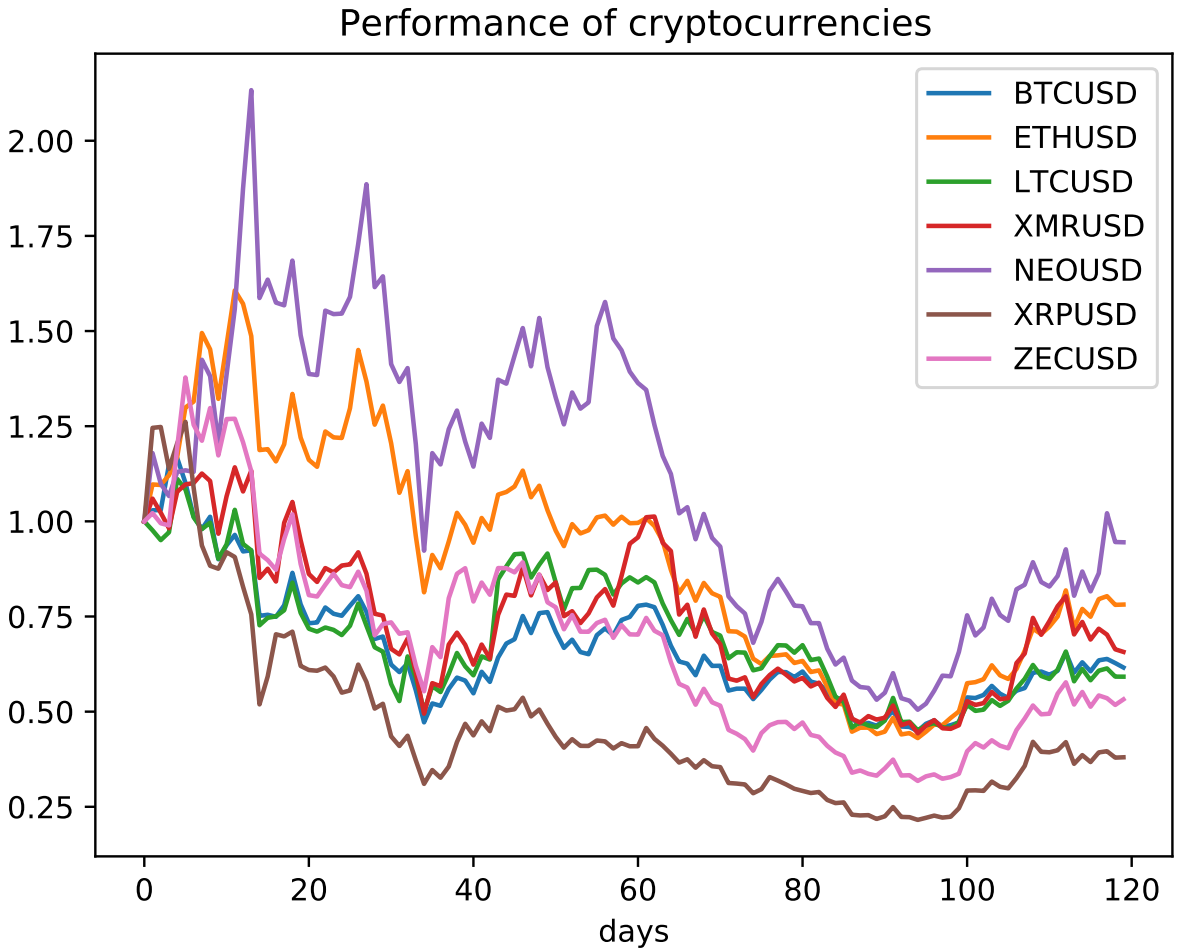
- There are two major approaches to determining the presence of cointegration of two or more time series:
  1. Granger-Engle
  2. Johansen
- Granger-Engle Method

  Represent one time series as a linear combination of the others:

$$X_{jt} = \sum_{i \neq j} \beta_i X_{it} + \epsilon_t.$$

- Fit using least squares and test whether the residuals are stationary using Dickey-Fuller
- If the residuals are stationary, then the cointegrating vector is $(\beta_1, \ldots, \beta_{j-1}, -1, \beta_{j+1}, \ldots \beta_m)$. There can be several cointegrating vectors.

Cryptocurrency



Example: 1

```
import quandl
import pandas as pd
import matplotlib.pyplot as plt
import requests
import statsmodels.tsa.stattools as ts
from statsmodels.tsa.vector_ar.vecm import coint_johansen
def get_bitfinex_asset(asset, ts_ms_start, ts_ms_end):
      url = 'https://api.bitfinex.com/v2/candles/trade:1D:t' + asset +
'/hist'
    params =  'start': ts_ms_start, 'end': ts_ms_end, 'sort': 1
    r = requests.get(url, params = params)
    data = r.json()
    return pd.DataFrame(data)[2]
```

Example: 2

```
start_date = 1514768400000 # 1 January 2018, 00:00:00
end_date = 1527811199000 # 31 May 2018, 23:59:59
assets = ['BTCUSD', 'ETHUSD', 'LTCUSD', 'XMRUSD', 'NEOUSD',
'XRPUSD', 'ZECUSD']
#Bitcoin, Ethereum, Litecoin, Monero, NEO, Ripple XRP, Zcash
crypto_prices = pd.DataFrame()
for a in assets:
    print('Downloading ' + a)
    crypto_prices[a] = get_bitfinex_asset(asset = a, ts_ms_start=
    start_date, ts_ms_end = end_date)
crypto_prices.head()
```

Example: 3

```
# Normalize prices by first value
norm_prices = crypto_prices.divide(crypto_prices.iloc[0])
plt.figure(figsize = (15, 10))
plt.plot(norm_prices)
plt.xlabel('days')
plt.title('Performance of cryptocurrencies')
plt.legend(assets)
plt.show()
for a1 in crypto_prices.columns:
        for a2 in crypto_prices.columns:
                if a1 != a2:
                test_result = ts.coint(crypto_prices[a1], crypto_prices[a2])
                print(a1 + ' and ' + a2 + ': p-value = ' +
str(test_result[1]))
```

Example: 4

BTCUSD and ETHUSD: p-value = 0.065769798042690 5
BTCUSD and LTCUSD: p-value = 0.0734714067845 1415
BTCUSD and XMRUSD: p-value = 0.0215708894241 8218
BTCUSD and NEOUSD: p-value = 0.1023948341904 1967
BTCUSD and XRPUSD: p-value = 0.0090012245739 9112
BTCUSD and ZECUSD: p-value = 0.163781282448 07383
ETHUSD and LTCUSD: p-value = 0.60975825185 0134
ETHUSD and XMRUSD: p-value = 0.172846430884 27865
ETHUSD and NEOUSD: p-value = 0.12876967722 06111
ETHUSD and XRPUSD: p-value = 0.83537247711 61406
ETHUSD and ZECUSD: p-value = 0.008516903095 807193

Example: 5

LTCUSD and XMRUSD: p-value = 0.17400005498761367
LTCUSD and NEOUSD: p-value = 0.1249504285029151
LTCUSD and XRPUSD: p-value = 0.3479861245950189 3
LTCUSD and ZECUSD: p-value = 0.220846004720878 5
XMRUSD and NEOUSD: p-value = 0.1590007249851493 3
XMRUSD and XRPUSD: p-value = 0.1053521818111652 9
XMRUSD and ZECUSD: p-value = 0.0868738183145278 4
NEOUSD and XRPUSD: p-value = 0.702811523154506 2
NEOUSD and ZECUSD: p-value = 0.1921763401357902 7
XRPUSD and ZECUSD: p-value = 0.693578179894564 6

Example: 6

```
cj = coint_johansen(crypto_prices, det_order = 0, k_ar_diff = 0)
cj.lr1
array([177.07125622, 112.17759288, 63.09167506,
40.91449977, 24.41337134, 10.94471608, 4.37697074])
cj.cvt
array([[120.3673, 125.6185, 135.9825],
       [ 91.109 , 95.7542, 104.9637],
       [ 65.8202, 69.8189, 77.8202],
       [ 44.4929, 47.8545, 54.6815],
       [ 27.0669, 29.7961, 35.4628],
       [ 13.4294, 15.4943, 19.9349],
       [ 2.7055, 3.8415, 6.6349]])
```

Example: 7

cj.lr2

```
array([64.89366334, 49.08591782, 22.17717529,
16.50112843, 13.46865526, 6.56774534, 4.37697074])
```

cj.cvm

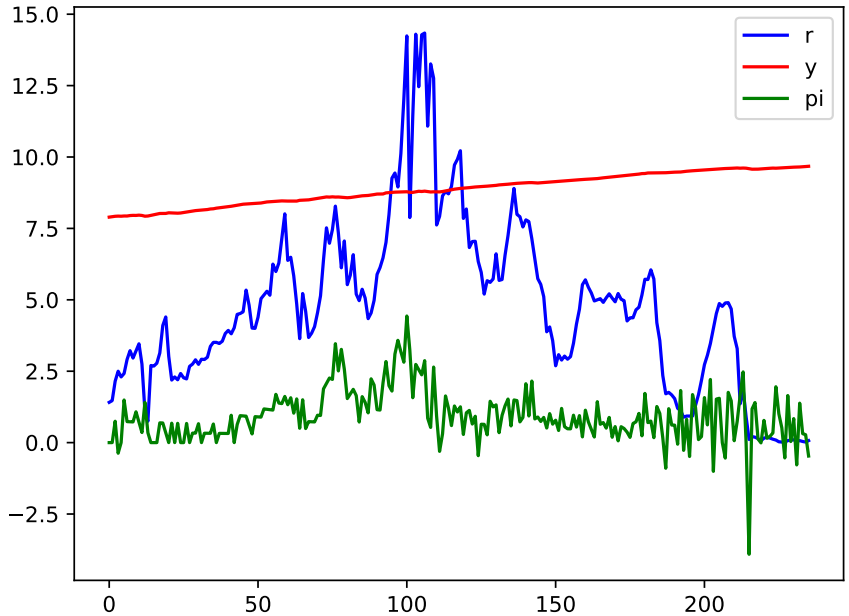
```
array([[43.2947, 46.2299, 52.3069],
       [37.2786, 40.0763, 45.8662],
       [31.2379, 33.8777, 39.3693],
       [25.1236, 27.5858, 32.7172],
       [18.8928, 21.1314, 25.865 ],
       [12.2971, 14.2639, 18.52 ],
       [ 2.7055, 3.8415, 6.6349]])
```

Example: 8

The data set contains three time series, r (the 91-day T-bill rate), y (the log of real GDP), and pi (the inflation rate). The goal is to perform an analysis to determine if any of these series are cointegrated.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from arch.unitroot import engle_granger
from arch.unitroot.cointegration import phillips_ouliaris
from statsmodels.tsa.vector_ar.vecm import coint_johansen
data = pd.read.csv('TbGdpPi.csv')
```

Example: 9



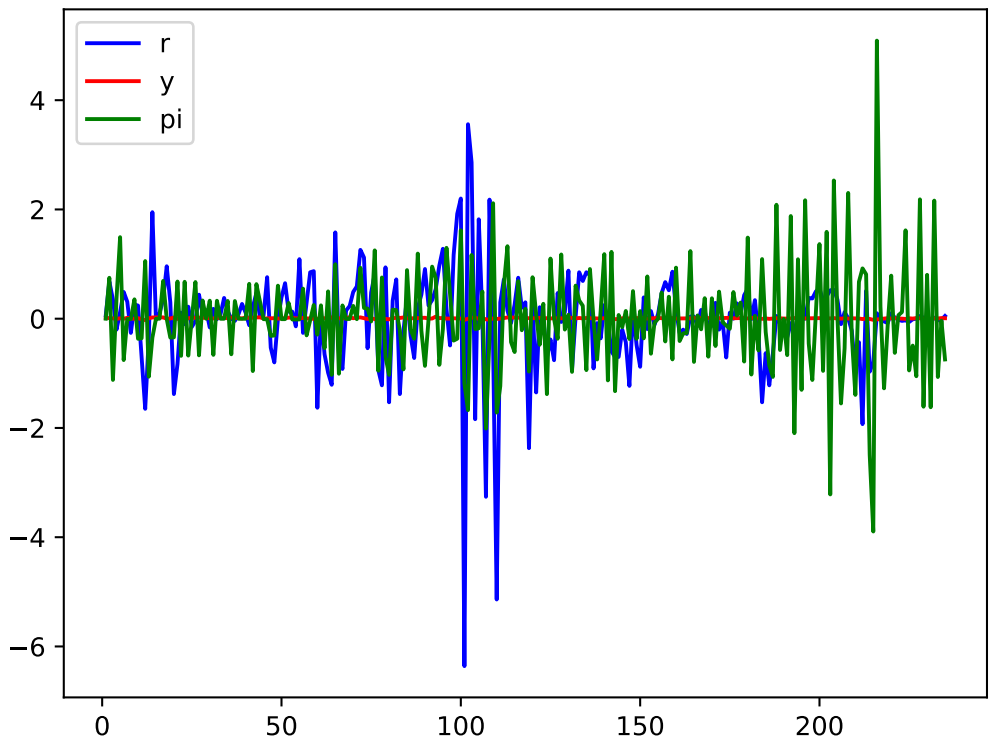

```
plt.plot(data['r'], 'b-', label = 'r')
plt.plot(data['y'], 'r-', label = 'y')
plt.plot(data['pi'], 'g-', label = 'pi')
plt.legend(loc = 'best')
plt.show()
```

Example: 10

- sm.tsa.stattools.adfuller(data['y'])
  (-1.45, 0.56, 2, 233, '1%': -3.46, '5%': -2.87, '10%': -2.57,)
  sm.tsa.stattools.adfuller(data['r'])
  (-1.58, 0.49, 12, 223, '1%': -3.46, '5%': -2.87, '10%': -2.57,)
  sm.tsa.stattools.adfuller(data['pi'])
  (-1.78, 0.39, 15, 220, '1%': -3.46, '5%': -2.87, '10%': -2.57,)
- All are nonstationary and H_0 (unit root) is clearly not rejected.
- diffdata = data.diff()[1:]

Example: 11

```
plt.plot(data_diff['r'], 'b-', label = 'r')
plt.plot(data_diff['y'], 'r-', label = 'y')
plt.plot(data_diff['pi'], 'g-', label = 'pi')
plt.legend(loc = 'best')
plt.show()
```



Example: 12

```
sm.tsa.stattools.adfuller(diffdata['y'])
```

(-7.87, 4.89e-12, 1, 233, '1%': -3.46, '5%': -2.87, '10%': -2.57,)

```
sm.tsa.stattools.adfuller(diffdata['r'])
```

(-5.35, 4.25e-06, 11, 223, '1%': -3.46, '5%': -2.87, '10%': -2.57)

```
sm.tsa.stattools.adfuller(diffdata['pi'])
```

(-5.98, 1.87e-07, 14, 220, '1%': -3.46, '5%': -2.87, '10%': -2.57)

All three H_0 are rejected, so all are $\mathcal{I}(1)$

```
eg_test = engle_granger(data['r'], data['pi'],trend = 'n')
eg_test
```

Example: 13

Engle-Granger Cointegration Test (r, pi)
Statistic: -2.377
P-value: 0.119
Null: No Cointegration, Alternative: Cointegration
ADF Lag length: 3
Trend: c
Estimated Root: 0.856

```
eg_testry = engle_granger(data['r'],data['y'])
eg_testry
```

Engle-Granger Cointegration Test (r,y)
Statistic: -2.29
P-value: 0.378
Null: No Cointegration, Alternative: Cointegration
ADF Lag length: 3
Trend: c

Example: 14

```
po_zt_testrpi = phillips_ouliaris(data['r'],data['pi'], trend = 'n',
test_type = 'Zt')
po_zt_testrpi.summary()
```

```
Phillips-Ouliaris zt Cointegration Test
=======================================
Test Statistic          -11.755
P-value                   0.000
Kernel                 Bartlett
Bandwidth               12.323
---------------------------------

Trend: Constant
Critical Values: -2.48 (10%), -2.79 (5%), -3.39 (1%)
Null Hypothesis: No Cointegration
Alternative Hypothesis: Cointegration
Distribution Order: 2
```

Example: 15

```
po_zt_testry = phillips_ouliaris(data['r'],data['y'],trend =
'n',test_type = 'Zt')
po_zt_testry.summary()
```

Phillips-Ouliaris zt Cointegration Test

==========

Test Statistic	-2.051
P-value	40.217
Kernel	Bartlett
Bandwidth	4.313

Trend: Constant

Critical Values: -2.48 (10%), -2.79 (5%), -3.39 (1%)

Null Hypothesis: No Cointegration

Alternative Hypothesis: Cointegration

Distribution Order: 2

Example: 16

```
po_zt_testpiy = phillips_ouliaris(data['pi'],data['y'], trend =
'n',test_type = 'zt')
po_zt_testpiy.summary()
```

```
Phillips-Ouliaris zt Cointegration Test
=======================================
Test Statistic          -11.880
P-value                   0.000
Kernel                  Bartlett
Bandwidth                11.959
---------------------------------

Trend: Constant
Critical Values: -2.48 (10%), -2.79 (5%), -3.39 (1%)
Null Hypothesis: No Cointegration
Alternative Hypothesis: Cointegration
Distribution Order: 2
```

Example: 17

```
johansen_rst=coint_johansen(data,det_order=0, k_ar_diff=0)
johansen_rst.lr1
  #tracestatistic
([131.27584764, 10.78357386, 4.18281327])
johansen_rst.lr2     # max eigenvalue statistic
([120.49227378, 6.60076059, 4.18281327])
johansen_rst.Results()
johansen_rst.cvt, critical values (.90,.95,.99) trace
((27.0669, 29.7961, 35.4628),
(13.4294, 15.4943, 19.9349),
( 2.7055, 3.8415, 6.6349))
johansen_rst.cvm critical values (.90.95,.99) max eigenvalue
((18.8928, 21.1314, 25.865 ),
(12.2971, 14.2639, 18.52 ),
( 2.7055, 3.8415, 6.6349))
```

Example: 18

```
trace_rst = 'Reject': list(johansen_rst.lr1 >
johansen_rst.cvt[:,1])
trace_rst = pd.DataFrame(trace_rst)
trace_rst.index = ['r', 'y', 'pi']
trace_rst
```

Example: 19

```
max_eigv_rst
= 'Reject': list(johansen_rst.lr2 > johansen_rst.cvm[:,1])
max_eigv_rst = pd.DataFrame(max_eigv_rst)
max_eigv_rst.index = ['r', 'y', 'pi']
max_eigv_rst
```

State Space Modeling: 1

- State space modeling Has a long history and is connected to important technology problems (e.g. Hidden Markov Models, which have been centrol in speech and image recognition)
- The idea is that there is a stochastic process which described the true state of a system (the state model), $\{\mathbf{X}_t,\ t \geq 0\}$.
- There is a second time series that represents the observation of this state process (the observation process), $\{\mathbf{Y}_t,\ t \geq 0\}$. These can be thought of as a noisy version of the true state process.
- The models can be univariate or multivariate, and are parameterized like the following:

$$\mathbf{X}_{t+1} = \mathbf{F}_t\mathbf{X}_t + \mathbf{V}_t,$$

$$\mathbf{Y}_t = \mathbf{G}_t\mathbf{X}_t + \mathbf{W}_t,$$

State Space Models: 2

- The two noise processes, $\mathbf{V}_t$ and $\mathbf{W}_t$ are assumed to be $\text{WN}(\mathbf{0}, \mathbf{\Sigma}_{X,t})$ and $\text{WN}(\mathbf{0}, \mathbf{\Sigma}_{Y,t})$ respectively.
- The two noise processes are uncorrelated, i.e. $E(\mathbf{V}_s \mathbf{W}_t) = \mathbf{0}, \ \forall s, t.$
- The standard time series models (AR, MA, ARMA) have state space representations.

State Space Models: 3

For example, consider an AR(2) model with mean 0.

$$X_t = \phi_1 X_{t-1} + \phi_2 X_{t-2} + \epsilon_t.$$

Define $\mathbf{X}_t = (X_t, X_{t-1})^T$, $\boldsymbol{\epsilon}_t = (\epsilon_t, 0)^T$,

$$\mathbf{X}_t = \mathbf{F}\mathbf{X}_{t-1} + \boldsymbol{\epsilon}_t$$

$$Y_t = (1, 0)\mathbf{X}_t.$$

with

$$\mathbf{F} = \begin{bmatrix} \phi_1 & \phi_2 \\ 1 & 0 \end{bmatrix}.$$

State Space Models: 4

Consider a MA(2) model with mean 0

$$X_t = \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \epsilon_t.$$

Define $\mathbf{X}_t = (\epsilon_t, \epsilon_{t-1}, \epsilon_{t-2})^T$, $\boldsymbol{\epsilon}_t = (\epsilon_t, 0, 0)^T$,

$$\mathbf{X}_t = \mathbf{F}\mathbf{X}_{t-1} + \boldsymbol{\epsilon}_t$$

$$Y_t = (1, \theta_1, \theta_2)\mathbf{X}_t,$$

with

$$\mathbf{F} = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

State Space Models: 5

- The state space modeling form is very flexible and allows for the expression of a wide range of models. There are also three distinct types of problems that arise in this context, each solved by Kalman Recursions: prediction/forecasting, filtering, and smoothing.
- Suppose there is an unknown, latent state vector process $\mathbf{X}_t$. We have noisy observations of this state vector, $\mathbf{Y}_t$ and some initial state Y_0. We want to use the observations, $\mathbf{Y}_t$, to make some inference about $\mathbf{X}_t$ of the following type:

State Space Models: 6

- **Prediction or Forecasting**
  Estimate $\mathbf{X}_{t+1}$ or $\mathbf{X}_{t+k}$ using $\mathbf{Y}_0, \mathbf{Y}_1, \cdots, \mathbf{Y}_t$. This can be denoted $\hat{\mathbf{X}}_{(t+k)|t}$
- **Filtering**
  Estimate $\mathbf{X}_t$ using $\mathbf{Y}_0, \mathbf{Y}_1, \cdots, \mathbf{Y}_t$. We denote this $\hat{\mathbf{X}}_{t|t}$
- **Smoothing**
  Estimate $\mathbf{X}_t$ given $\mathbf{Y}_1, \mathbf{Y}_2, \cdots, \mathbf{Y}_T$, for $1 \leq t < T$. This is denoted $\hat{\mathbf{Y}}_{t|T}$ indicating the estimate of $\mathbf{X}_t$ conditional on $\mathcal{F}_T$.

State Space Models: 7

- In all three cases there are **Kalman recursions** to solve these problems based on minimizing mean square error. The recursions allow one to solve these problems “on-line,” that is in the case of forecasting and filtering to use the current results and the new data to obtain new estimates/forecasts without needing the entire data set that preceeded it.

Kalman Prediction: 1

- In Kalman prediction, the goal is to predict/forecast $\mathbf{X}_{t+1}$ given $(\mathbf{Y}_0, \mathbf{Y}_1, \cdots, \mathbf{Y}_t)$, a future value of $\mathbf{X}$ given the information from $\mathbf{Y}$ up to time t.
- We let the mean squared error of the prediction be denoted by

$$\Omega_t = E((\mathbf{X}_{t+k} - \hat{\mathbf{X}}_{t+k})(\mathbf{X}_{t+k} - \hat{\mathbf{X}}_{t+k})^T).$$

- The Kalman prediction recursion begins with $\hat{\mathbf{X}}_1$ and proceeds one-step at a time to $\hat{\mathbf{X}}_{t+1}$, as follows:

Kalman Prediction: 2

$$\hat{\mathbf{X}}_1 = E(\mathbf{X}_1 | \mathbf{Y}_0),$$

$$\hat{\mathbf{X}}_{t+1} = \mathbf{F}_t \hat{\mathbf{X}}_t + \mathbf{\Theta} \mathbf{\Delta}_t^{-1} (\mathbf{Y}_t - \mathbf{G}_t \hat{\mathbf{X}}_t),$$

where

$$\mathbf{\Delta}_t = \mathbf{G}_t \mathbf{\Omega}_t \mathbf{G}_t^T + \mathbf{R}_t,$$

$$\mathbf{\Theta}_t = \mathbf{F}_t \mathbf{\Omega}_t \mathbf{G}_t^T.$$

A Simple Example: 1

- Consider a simple linear regression model

$$Y_t = bZ_t + \epsilon_t,$$

where ϵ_t are i.i.d. $N(0, \sigma^2)$ and the predictor Z_t is known.

- We first express this as a state-space model with the goal being to estimate the unknown parameter b:

$$X_t = b,$$

$$Y_t = G_t X_t + W_t,$$

where $G_t = Z_t$, $R_t = \sigma^2, F = 1, Q = 0$

- Estimate X_t given $Y_i,\ i \leq t$.

A Simple Example: 2

- We know the answer from simple least squares

$$\hat{b}_t = \sum_{i=1}^{t} Y_i Z_i / \sum_{i=1}^{t} Z_i^2.$$

- How can this come from the Kalman equations, and how can $\hat{b}_t$ be computed recursively?

$$\begin{aligned}
\hat{b}_t &= \frac{\sum_{i=1}^{t-1} Z_i^2}{\sum_{i=1}^{t} Z_i^2} \frac{\sum_{i=1}^{t-1} Y_i Z_i + Y_t Z_t}{\sum_{i=1}^{t-1} Z_i^2} \\
&= \frac{\sum_{i=1}^{t-1} Z_i^2}{\sum_{i=1}^{t} Z_i^2} (\hat{b}_{t-1} + \frac{Y_t Z_t}{\sum_{i=1}^{t-1} Z_i^2}) \\
&= next
\end{aligned}$$

A Simple Example: 3

$$\begin{aligned}\hat{b}_t &= \hat{b}_{t-1} - \frac{Z_t^2}{\sum_{i=1}^t Z_i^2}\hat{b}_{t-1} + \frac{Y_t Z_t}{\sum_{i=1}^t Z_i^2} \\ &= \hat{b}_{t-1} + \frac{Z_t}{\sum_{i=1}^t Z_i^2}(Y_t - Z_t\hat{b}_{t-1})\end{aligned}$$

- The quantity $K_t = Z_t / \sum_{i=1}^t Z_i^2$, the "Kalman Gain."
- $$Var(\hat{b}_t) = (1 - K_t Z_t) Var(\hat{b}_{t-1}).$$

Kalman Filtering

- For Kalman filtering, we want to predict/estimate the current state, $\mathbf{X}_t$ given $(\mathbf{Y}_0, \cdots, \mathbf{Y}_t)$. This is denoted by $\mathbf{X}_{t|t}$.
- The estimate is a combination of the one-step-ahead prediction from time $t-1$, $\hat{\mathbf{X}}_t$ and the difference between the observed $\mathbf{Y}_t$ and its predicted value using the observation equation, $\mathbf{Y}_t - \mathbf{G}_t\hat{\mathbf{X}}_t$

$$\mathbf{X}_{t|t} = \hat{\mathbf{X}}_t + \Omega_t \mathbf{G}_t^T \mathbf{\Delta}_t^{-1}(\mathbf{Y}_t - \mathbf{G}_t\hat{\mathbf{X}}_t),$$

where the relevant terms were defined earlier.

Stochastic Volatility: 1

- Recall we have develop ARCH/GARCH modeling to create models that better conform to the stylized facts of financial markets: heteroscedasticity, volatility clustering, heavy tail behavior.
- The GARCH model created this behavior “within itself,” not based on external events/shocks. Yet we have seen that many stocks (particularly those impacted by the same factors, such as the market, industry sectors, etc) exhibit similar behavior with respect to volatility clutering.
- An alternative model would allow for exogenous processes to impact the volatility behavior of a financial asset.
- Such models are called stochastic volatility models, and a state space formulation of these models allows us to employ “Kalman methodology” to filter and predict.

Stochastic Volatility: 2

- In state space models, there is an unobserved or latent variable(s) modeled and an observation/measurement process. We often want to make inference about the latent process.
- A very good example of this situation is volatility. Volatility is unobserved, yet crucial to the valuation of assets (e.g. an option price).
- We want to model volatility as the unobserved state and then have an observation process that can be used to estimate (filter) and predict the latent process.
- An example using the Heston model is presented in the paper "Filtering in Finance."

Stochastic Volatility: 3

- The Heston model involves the state process, v_t, and the observation process, S_t, defined by the SDE:

$$dv(t) = (\omega - \theta v_t)dt + \xi\sqrt{v_t}dW_t^v,$$

$$dS_t = rS_t dt + \sqrt{v(t)}S_t dW_t^S.$$

- The volatility process is a CIR process, whereas the stock price process is GBM with stochastic volatility where:
  - r is the risk free rate,
  - θ is the speed of mean reversion,
  - ω/θ is the long-run equilibrium variance
  - ξ is the volatility of the variance process.
  - ρ is the correlation between W_t^v and W_t^S
- Conditional on v_t, the price process is a GBM

Stochastic Volatility: 4

- The Heston model is defined in continuous time, and we need to convert it to discrete time, and put it into state space form.
- The parameter set $(\omega, \theta, \xi, \rho)$ must be calibrated with market data to have the model in risk-neutral form.
- We can rewrite the stock price equation

$$\log S_{t+\Delta} = \log S_t + (r - .5v_t)\Delta + \sqrt{v_t \Delta} Z_t^S.$$

- We next use an Euler discretization for the two SDEs with time step Δ:

Stochastic Volatility: 5

$$\log S_{(i+1)\Delta} = \log S_{i\Delta} + (r_{i\Delta} - .5 v_{i\Delta})\Delta + \sqrt{v_{i\Delta}\Delta} Z^S_{(i+1)\Delta},$$

$$v_{(i+1)\Delta} = v_{i\Delta} + (\omega - \theta v_{i\Delta})\Delta + \xi\sqrt{v_{i\Delta}\Delta} Z^v_{(i+1)\Delta},$$

where $\{(Z_i^S, Z_i^v),\ 1 \le i \le N\}$, $\Delta = T/N$, are iid standard bivariate normal random vectors with correlation ρ.

- It is possible to eliminate the correlation parameter ρ by introducing the independent pair $(\tilde{Z}_i^v, Z_i^S)$ where

$$Z_i^v = \rho Z_i^S + \sqrt{1-\rho^2}\tilde{Z}_i^v$$

and substituting $\tilde{Z}^v_{i\Delta} = \frac{1}{\sqrt{1-\rho^2}}(Z^v_{i\Delta} - \rho \tilde{Z}^S_{i\Delta})$ into the volatility equation.

Stochastic Volatility: 6

- Substituting $\sqrt{v_k \Delta} Z_i^S = \log(S_{(i+1)\Delta}/S_{i\Delta}) - (r_{i\Delta} - .5 v_{i\Delta})\Delta$ into the volatility equation, we arrive at the state space version of the Euler discretized Heston model with independent noise terms:

$$\begin{aligned} v_{(i+1)\Delta} &= v_{i\Delta} + [\omega - \rho\xi r_{i\Delta} - (\theta - .5\rho\xi) v_{i\Delta}]\Delta \\ &+ \rho\xi \log(S_{(i+1)\Delta}/S_{i\Delta}) + \xi\sqrt{1-\rho^2}\sqrt{v_{i\Delta}\Delta}\tilde{Z}_k^v. \end{aligned}$$

for the state equation and

$$\log S_{(i+1)\Delta} = \log S_{i\Delta} + (r_{i\Delta} - .5 v_{i\Delta})\Delta + \sqrt{v_{i\Delta}\Delta} Z_k^S,$$

for the measurement equation.

Stochastic Volatility: 7

- Once the stochastic volatility model is in state space form, we can utilize the Kalman procedures for statistical inference.
- In the particular case, the volatility is of great interest, and inferring the current volatiity using Kalman filtering has significant implications (e.g. in valuation of options, comparing current volatility estimates with implied volatility)
- There are a number of stochastic volatility models, e.g. the Hull-White model we used in the simulation course.
- Estimation of these models is known to be more complicated that GARCH models.